TESSERA

The permission layer for AI agents

Working Draft v0.6

Guglielmo Reggio

April 2026

The Roman tessera hospitalis *was a token split between two strangers.*
Reuniting the pieces proved trust across distance.

Contents

1 The Problem **2**

1.1 Agents now have real-world power . . . 2

1.2 Identity is not enough . . . 2

1.3 The missing primitive . . . 3

2 What Tessera Is **3**

2.1 Human credential . . . 3

2.2 Agent credential . . . 3

3 The First Product: Tessera Guard **4**

3.1 Guarded action classes . . . 4

3.2 Verification flow . . . 4

3.3 Why Guard matters . . . 4

4 Design Principles **5**

4.1 Execution-time authority . . . 5

4.2 Scoped delegation . . . 5

4.3 Fast revocation . . . 5

4.4 Minimal disclosure . . . 5

4.5 Runtime portability . . . 5

4.6 Open protocol, not closed platform lock-in . . . 5

5 Protocol Model **5**

5.1 Layer 1 — Human root credential . . . 6

5.2 Layer 2 — Agent identity . . . 6

5.3 Layer 3 — Delegation . . . 6

5.4 Layer 4 — Verification . . . 6

6 Credential Structure **7**

6.1 Human credential . . . 7

6.2 Agent credential . . . 7

6.3 Containment rule . . . 7

7 Revocation **8**

8 Privacy Model **8**

9 Threat Model **8**

10 Integration Surfaces **9**

10.1 Agent runtimes 9

10.2 MCP servers and gateways 9

10.3 Developer platforms and APIs 9

10.4 Enterprise internal agents 9

11 Why Now **9**

11.1 Agents are crossing from assistance into action 10

11.2 Security models are lagging 10

11.3 Standards are still open 10

12 Go-to-Market **10**

12.1 Phase 1 — OpenClaw and open agent ecosystems 10

12.2 Phase 2 — Cross-runtime expansion 10

12.3 Phase 3 — Enterprise control plane 11

13 Business Model **11**

14 Non-goals for the Initial Version **11**

15 Roadmap **12**

16 Why the Name Tessera **12**

17 Conclusion **12**

18 Get Involved **13**

Abstract

AI agents can now send messages, execute code, spend money, publish content, and operate tools across the web. But the internet still lacks a standard way to answer four questions at the moment an agent acts: who authorised this agent, what is it allowed to do, when does that authority expire, and can that authority be revoked immediately?

Tessera is an open protocol for execution-time authorisation of AI agents. It makes sensitive agent actions contingent on a valid, human-delegated credential. Before an agent can send an email, trigger a payment, execute a shell command, or publish content, it must present cryptographic proof that a real human authorised that class of action within a defined scope and time window.

Tessera's first product is Tessera Guard: a runtime middleware layer that sits between agent frameworks and sensitive tools, verifies agent credentials, and blocks unauthorised actions by default.

Under the hood, Tessera combines a human root credential, privacy-preserving presentation, scoped delegation from human to agent, revocation and policy verification at execution time, and an open SDK for agent runtimes, tool gateways, and platforms.

Tessera is not a cryptocurrency, does not depend on biometric hardware, and does not require platforms to learn user identity. Its purpose is narrower and more practical: to become the standard authorisation layer for AI agents.

1. The Problem

1.1 Agents now have real-world power

AI systems are no longer limited to generating text. They increasingly act through agents that can:

- send emails and messages,
- modify code and infrastructure,
- call APIs and external services,
- make purchases and subscriptions,
- post public content,
- and coordinate with other agents.

This is a structural shift. Once an agent can act, the key question is no longer just whether it is intelligent. The key question is whether it is authorised.

1.2 Identity is not enough

Traditional identity systems answer questions about who a user is. They do not cleanly answer what an autonomous agent is allowed to do on that user's behalf at the moment of execution.

Existing systems rely on one of three flawed patterns:

1. **Ambient authority** — if the agent has access to a tool or token, it can act.
2. **Repeated approval prompts** — the human is asked every time, destroying the utility of autonomy.
3. **Platform-specific permissions** — each runtime reinvents authorisation in its own format.

None of these creates portable, auditable, revocable delegation.

1.3 The missing primitive

The internet needs a primitive for human-to-agent delegated authority.

That primitive must allow a verifier to check, in real time:

- that a human principal exists,
- that the human delegated authority to this specific agent,
- that the delegated scope covers the requested action,
- that the authorisation is still valid,
- and that it has not been revoked.

That is the problem Tessera solves.

2. What Tessera Is

Tessera is an open authorisation protocol for agentic systems.

It introduces two credential types.

2.1 Human credential

A human credential proves that a real human principal exists and is eligible to delegate authority.

2.2 Agent credential

An agent credential is issued by that human to an agent, defining:

- the agent identity,
- the permitted actions,
- optional spending or usage limits,
- optional resource constraints,
- the issuance time,

- the expiry time,
- and the human signature authorising the delegation.

The protocol is designed so that sensitive actions are checked at execution time, not assumed from prior login state or API possession.

3. The First Product: Tessera Guard

Tessera Guard is the first implementation of the protocol.

It is a middleware layer that intercepts sensitive tool calls and blocks them unless the agent presents a valid Tessera credential.

3.1 Guarded action classes

The initial protected classes are:

- `message.send`
- `payment.intent`
- `exec.shell`
- `content.publish`

These classes are deliberately narrow. They cover high-risk actions with clear user harm and immediate practical demand.

3.2 Verification flow

When an agent attempts a guarded action:

1. The runtime passes the requested action and attached credential to Tessera Guard.
2. Guard verifies credential validity.
3. Guard verifies issuer and delegation signatures.
4. Guard checks expiry.
5. Guard checks revocation status.
6. Guard checks whether the requested action falls within delegated scope.
7. Guard returns allow or deny, with a structured explanation.

This turns authorisation into a deterministic runtime boundary.

3.3 Why Guard matters

Tessera Guard changes the default posture of agent systems:

- without Guard: agents act unless stopped
- with Guard: agents cannot act unless authorised

That inversion is the core product value.

4. Design Principles

Tessera is designed around the following principles.

4.1 Execution-time authority

Authority must be evaluated when the action executes, not inferred from long-lived access.

4.2 Scoped delegation

An agent can act only within the authority explicitly granted to it.

4.3 Fast revocation

A human must be able to invalidate delegated authority immediately.

4.4 Minimal disclosure

Verifiers should learn only what they need to know to authorise the action.

4.5 Runtime portability

Authorisation should work across agent frameworks, MCP servers, gateways, and platforms.

4.6 Open protocol, not closed platform lock-in

The core credential and verification model should be implementable by anyone.

5. Protocol Model

Tessera has four layers.

5.1 Layer 1 — Human root credential

A human obtains a root credential from an issuer. This establishes eligibility to delegate authority.

The exact trust anchor may vary by deployment and jurisdiction. In early versions, Tessera may rely on existing verification rails such as regulated identity or financial verification providers. The protocol does not require revealing personal identity to verifiers during runtime authorisation.

5.2 Layer 2 — Agent identity

Each agent has its own public key and stable identifier.

An agent is not merely a session. It is a cryptographic subject capable of receiving delegated authority.

5.3 Layer 3 — Delegation

The human signs a delegation to the agent. The delegation includes:

- agent public key or identifier,
- allowed action classes,
- optional quantitative limits,
- optional contextual constraints,
- issuance time,
- expiry time,
- revocation handle,
- human signature.

5.4 Layer 4 — Verification

At runtime, a verifier checks:

- credential integrity,
- signature chain,
- scope match,
- expiration,
- revocation,
- and policy constraints.

The result is a simple authorisation decision.

6. Credential Structure

6.1 Human credential

A Tessera human credential attests that the holder is authorised to delegate agent authority.

The verifier should not need to know the person's name, bank, email, or legal identity to evaluate a delegated action. The runtime only needs to know whether the human root is valid and what assurance tier or policy class it carries.

6.2 Agent credential

A Tessera agent credential contains:

- a reference to the parent human root,
- the agent identifier,
- the delegation scope,
- issuance and expiry,
- and the parent signature.

Example shape:

```
{
  "type": "TesseraAgentCredential",
  "agent": "agent_pk_123",
  "parent": "human_commitment_abc",
  "scope": {
    "actions": ["message.send", "content.publish"],
    "max_recipients": 20,
    "allowed_domains": ["example.com"]
  },
  "issued_at": 1743465600,
  "expires_at": 1743552000,
  "revocation_id": "rev_456",
  "signature": "..."
}
```

6.3 Containment rule

No child delegation may exceed the permissions of its parent.

This allows safe sub-delegation in multi-agent systems while preserving a clear authority chain.

7. Revocation

Revocation is a first-class requirement.

If a human revokes an agent credential, the next guarded action must fail immediately.

Tessera therefore treats revocation not as a slow administrative process, but as an online part of authorisation.

A production deployment may use a combination of:

- status registries,
- signed status lists,
- cached freshness windows,
- and high-availability revocation endpoints.

The exact implementation can evolve, but the product requirement is fixed:

> revocation must be fast enough to function as a real kill switch.

8. Privacy Model

Tessera aims to minimise disclosure during verification.

A verifier should learn:

- whether the credential is valid,
- whether the action is within scope,
- when the credential expires,
- and what policy tier applies.

A verifier should not need to learn:

- the human's civil identity,
- the underlying verification source,
- unrelated activity history,
- or linkable information across unrelated verifications.

Privacy is not the entire product, but it matters. Authorisation for agents should not require turning every delegated action into a full identity disclosure event.

9. Threat Model

Tessera is designed to reduce the following risks:

Risk	Defence
Unauthorised action	Execution-time scope checks.
Credential replay	Bounded validity, agent-bound keys, and revocation.
Scope escalation	Deterministic scope evaluation and containment rules.
Multi-agent ambiguity	Signed delegation chain from human root to acting agent.
Opaque failure	Structured denial reasons that agents can surface to users.

Table 1: Core Tessera threat model.

Tessera does not solve every misuse problem. A real human may still authorise harmful behaviour. Tessera's role is narrower: it makes authority explicit, bounded, and auditable.

10. Integration Surfaces

Tessera is designed for four initial integration surfaces.

10.1 Agent runtimes

Open-source and commercial runtimes can add Guard to their tool invocation path.

10.2 MCP servers and gateways

Servers that expose tools to agents can require Tessera credentials before serving sensitive operations.

10.3 Developer platforms and APIs

Services can distinguish between unauthenticated automation and human-authorised agent actions.

10.4 Enterprise internal agents

Organisations can require bounded delegation for agents acting in internal systems.

11. Why Now

Three trends make this the right moment.

11.1 Agents are crossing from assistance into action

The risk surface has moved from text generation to real-world execution.

11.2 Security models are lagging

Most agent systems still rely on ambient tool access, brittle prompts, or repeated approval loops.

11.3 Standards are still open

The market is early enough that the authorisation layer for agents has not yet ossified into a single closed vendor model.

This creates room for an open protocol to define the category.

12. Go-to-Market

Tessera starts where the pain is highest and the feedback loop is fastest.

12.1 Phase 1 — OpenClaw and open agent ecosystems

The first battlefield is the open agent developer community.

Why:

- these users already understand tools and permissions,
- they can adopt middleware quickly,
- they produce visible design feedback,
- and they are highly valuable for protocol iteration.

The goal of this phase is not maximum revenue. It is product truth.

12.2 Phase 2 — Cross-runtime expansion

Once validated in one ecosystem, Tessera expands to:

- MCP-compatible tools,
- agent gateways,
- coding agents,
- workflow automations,
- and API services receiving agent traffic.

12.3 Phase 3 — Enterprise control plane

The long-term commercial layer includes:

- hosted verification,
- revocation infrastructure,
- policy administration,
- audit trails,
- team and org controls,
- and enterprise integrations.

This creates a natural model:

- open protocol,
- open-source Guard,
- managed control plane.

13. Business Model

Tessera's commercial model should remain tightly aligned to authorisation infrastructure.

The clearest monetisable services are:

- hosted verification,
- revocation infrastructure,
- policy management,
- enterprise auditability,
- and premium support.

This keeps the business legible. The protocol can remain open while the operational control plane becomes the commercial offering.

Early versions should resist adding unnecessary monetary complexity. The main value is not speculative economics. The main value is safe delegated execution.

14. Non-goals for the Initial Version

Tessera is not, in its initial form:

- a universal social network identity,
- a cryptocurrency,
- a consumer payments system,

- a full legal identity replacement,
- or a generalised provenance system for all digital content.

Those may become adjacent extensions over time, but they are not the launch product.

The launch product is narrower:

> authorisation for sensitive AI agent actions.

15. Roadmap

Phase	Scope
Phase 0 — Protocol and SDK	Credential schema. Signature and verification logic. Local demo verifier. Basic revocation model.
Phase 1 — Tessera Guard for OpenClaw	Tool interception middleware. Initial guarded action classes. Structured denial responses. Real developer feedback.
Phase 2 — Cross-runtime support	MCP and gateway integrations. Policy templates. Hosted verification service.
Phase 3 — Enterprise layer	Multi-user admin. Team delegation. Audit and compliance features. High-availability revocation and policy infrastructure.
Phase 4 — Broader standards participation	Standards engagement. Interop work. Multi-issuer ecosystem.

Table 2: Narrowed v0.6 roadmap.

16. Why the Name Tessera

A Roman *tessera hospitalis* was a token broken into matching parts between strangers. Reuniting the pieces established trust across distance.

That image captures the protocol well: human authority on one side, agent action on the other, joined only when the pieces match.

17. Conclusion

AI agents need more than prompts, approvals, or platform-specific permissions. They need a standard way to prove who authorised them, what they are allowed to do, and whether that authority still holds.

Tessera exists to provide that layer.

Its purpose is simple:

before an agent can act, it must be authorised.

That should become a standard property of the internet.

18. Get Involved

Tessera is looking for:

- early runtime integrations,
- security and identity reviewers,
- design partners in agent infrastructure,
- and contributors to the open SDK and Guard.

The protocol is open because agent authorisation should become infrastructure, not a silo.

GitHub: https://github.com/tessera-protocol/tessera

Tessera Working Draft v0.6 — April 2026 — Licensed under CC BY 4.0